Experiment 10 : MINI PROJECT Report On

Multi-Label Toxic Comment Detection and Automated Content Masking Using Hybrid NLP Techniques

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology in

Computer Science and Engineering (AI & ML)

ODISHA UNIVERSITY OF TECHNOLOGY AND RESEARCH, BHUBANESWAR

Submitted By:

Astha Dakhinray
Registration No. : **23110201**
Branch : **CSE (AI and ML)**
Semester : **6th**

Session: 2025-2026

CONTENT

Sl. No.	Topic	Page No.
1	Introduction	3
2	Literature Survey	7
3	Objective and Motivation	10
4	Proposed Model	12
5	Experimental Setup	17
6	Experimental Results	19
7	Result Discussion	24
8	Conclusions and Future Work	25
9	References	26

1. INTRODUCTION

1.1 Background and Context

The rapid growth of online platforms such as social media networks, discussion forums, video-sharing websites, and comment sections has fundamentally changed how people communicate and share information across the globe.[1] Platforms such as YouTube, Twitter, Reddit, and Instagram have collectively attracted billions of active users. According to a 2023 survey on hate speech detection published in the ACM Computing Surveys, the volume of user-generated content on these platforms has grown exponentially, making manual monitoring of harmful content practically impossible.[2][3] While these platforms have enabled unprecedented levels of connection and information sharing, they have also created environments where harmful and abusive language can spread with very little friction or consequence.[4] Toxic comments, hate speech, personal insults, threats, and obscene language have become serious problems affecting millions of users every day. Studies focusing on global populations show alarming patterns of online harassment. The psychological effects of sustained exposure to toxic online content include anxiety, depression, social withdrawal, and reduced willingness to participate in online communities.[5] These impacts are not limited to adults; younger users and minorities are particularly vulnerable, making automated content moderation a matter of both platform governance and public health.[6][7] Platforms have increasingly been held legally and socially accountable for the content they host, adding regulatory urgency to the technical challenge of automated detection.First, multi-label classification is a significantly more realistic formulation than binary classification for online toxic content detection, and every modern study evaluates against multi-label benchmarks.[1,8,14] Second, hybrid feature approaches combining TF-IDF with word embeddings consistently outperform either representation used alone, particularly for semantically complex or rare toxicity categories.[9,11,15] Third, class imbalance must be explicitly addressed through balanced weighting or oversampling to achieve meaningful recall on minority labels.[2,13] Fourth, automated content masking is an empirically validated post-detection intervention that reduces harm more effectively than outright deletion over time.[6] Fifth, despite the performance advantage of transformer-based models, classical machine learning approaches remain practical and deployable without GPU infrastructure.[3]

1.2 Evolution of NLP in Content Moderation

Traditional approaches to handling toxic content relied on manual moderation and simple rule-based keyword filtering. Human reviewers read and flagged harmful comments, but this is neither scalable nor consistent. Natural Language Processing (NLP) provides a principled computational framework for this challenge. [8] NLP is a branch of Artificial Intelligence focused on enabling computers to understand, interpret, and process human language. Over the past decade, NLP has advanced from hand-crafted rule lists to statistical models, and more recently to neural and transformer-based architectures.[1] A comprehensive 2023 systematic review on class imbalance in NLP classification tasks[2] demonstrated that classical machine learning methods combined with proper feature engineering remain highly competitive for text classification tasks, particularly when labeled data is limited and interpretability is important.[9] Despite the emergence of large transformer-based models, the literature consistently shows that TF-IDF combined with word embeddings provides a strong and resource-efficient baseline for toxicity detection. More recent transformer-based approaches such as BERT-based models have achieved state-of-the-art results in many NLP benchmarks, as shown by Mozafari et al.[10] in their 2023 study on BERT-based hate speech detection. However, these models require significant computational resources that are not accessible in all academic or deployment settings. This has maintained the relevance and practical value of classical approaches such as those implemented in this project.

1.3 Problem Statement

The problem of toxic comment detection is considerably more complex than it appears. Toxicity is context-dependent, culturally variable, and multi-dimensional. Most existing production systems treat the problem as a simple binary classification: toxic or not toxic. This formulation is a serious oversimplification that fails in practice. A comment might be mildly offensive in isolation but constitute a targeted threat within a conversation. Coded language, sarcasm, and deliberate misspelling are routinely used to evade binary classifiers. Furthermore, the multi-label nature of real toxic content is ignored by binary systems. A comment can simultaneously be obscene, insulting, and identity-based hate speech. Knowing which specific dimensions of toxicity are present allows platform administrators to apply a proportionate and legally defensible moderation response. Earlier

systems that used only simple word frequency features such as TF-IDF vectors often failed to understand context and semantic meaning, resulting in high rates of both false positives and false negatives.[9]

1.4 Proposed Solution

This project proposes a ***comprehensive Multi-Label Toxic Comment Detection and Automated Content Masking system***. The system uses a hybrid feature extraction approach combining TF-IDF with Word2Vec word embeddings, capturing both statistical term importance and semantic similarity.[11] It classifies comments into six distinct toxicity categories simultaneously using a *MultiOutputClassifier* architecture, compares three classification algorithms to select the best performer through empirical evaluation, and implements a three-level automated content masking feature that hides toxic terms from public view while preserving the surrounding context of the comment. The system is designed to be computationally accessible, running entirely on CPU without requiring GPU hardware. This makes it deployable in academic and resource-constrained production environments. The Jigsaw Toxic Comment Classification dataset, which contains 159,571 manually labeled Wikipedia comments and is one of the most widely referenced benchmarks in this domain[8], is used as the training and evaluation corpus.

This project presents a comprehensive ***Multi-Label Toxic Comment Detection and Automated Content Masking System*** designed to address the growing challenge of moderating harmful online interactions. Unlike traditional binary classification systems that simply label content as toxic or non-toxic, this system performs **multi-label classification**, enabling it to detect and categorize multiple forms of toxicity within a single comment. Specifically, each comment can simultaneously be classified into six categories: toxic, severe toxic, obscene, threat, insult, and identity hate. This reflects the complex and overlapping nature of real-world abusive language, where a single comment may exhibit multiple harmful traits. [11]To achieve robust performance, the system employs a ***hybrid feature extraction strategy*** *that combines* ***TF-IDF (Term Frequency–Inverse Document Frequency)*** with **Word2Vec word embeddings**. TF-IDF captures the statistical importance of words within the corpus, effectively identifying keywords that are strongly associated with toxic behavior. However, relying solely on TF-IDF assumes that word importance is independent of context, which is a limitation when dealing with nuanced language. To address this, Word2Vec embeddings are incorporated to model semantic relationships between words, allowing the system to understand contextual similarity (e.g., recognizing that semantically similar insults may not share identical vocabulary). This combination ensures that both frequency-based relevance and contextual meaning are captured, leading to more accurate feature representation.For classification, the system utilizes a **MultiOutputClassifier architecture**, which extends traditional machine learning models to handle multiple output labels simultaneously. Instead of training a single classifier for each label independently, this approach allows a unified framework where multiple binary classifiers operate in parallel, improving scalability and consistency. The project further strengthens its methodological rigor by implementing and comparing three different classification algorithms (such as Logistic Regression, Naïve Bayes, and Support Vector Machines).

The model is trained and evaluated using the Jigsaw Toxic Comment Classification dataset, which consists of 159,571 Wikipedia talk page comments annotated by human evaluators. This dataset is one of the most widely used benchmarks in the field of toxic language detection, providing a reliable and diverse corpus for model development. Its multi-label structure and real-world origin make it particularly suitable for evaluating systems designed for practical content moderation tasks. [2,13] By combining detection with automated masking and ensuring computational accessibility, the project moves beyond theoretical modeling toward a solution that is both technically sound and practically deployable. [12]

1.5 Significance of the Study

This project is significant from three perspectives. From a technical perspective, it demonstrates a complete end-to-end NLP pipeline from raw text to deployment-ready predictions, incorporating both classical and modern NLP techniques within an interpretable framework.[12] From a social perspective, it directly addresses the growing problem of automated moderation of harmful content on public platforms. From an academic perspective, the project covers core NLP concepts including text preprocessing, hybrid feature extraction, multi-label classification, imbalanced learning, and comprehensive model evaluation , making it a thorough demonstration of applied NLP engineering skills. The three-level automated content masking system distinguishes this project from pure detection systems by providing an immediately deployable moderation action. Vidgen and Derczynski[3] emphasized in their 2024 study on abusive language training data that the quality of the complete pipeline , from data preprocessing to post-detection action , is what determines real-world impact. This project directly addresses all stages of that pipeline, from data cleaning to actionable output generation.

2 LITERATURE SURVEY

This chapter presents a structured review of existing research in text classification and toxic comment detection using Natural Language Processing. The survey traces the progression from early rule-based systems to classical machine learning and modern deep learning approaches, and identifies the gaps that motivated the design decisions of this project.

2.1 Early Approaches to Toxic Comment Detection

Early research on automated toxic content detection relied primarily on keyword blocklists and regular expression matching. These rule-based systems were brittle, easily defeated by misspelling, abbreviation, and coded language, and required constant manual updates. The transition to data-driven machine learning approaches was motivated by the need for systems that could generalize beyond hand-crafted rules. A comprehensive 2023 survey by Fortuna and Nunes[1] covering over 170 studies on automatic hate speech detection concluded that even simple machine learning classifiers trained on bag-of-words or TF-IDF features significantly outperformed all rule-based systems evaluated, while also noting that modern multi-label formulations are a more faithful representation of real online toxicity than binary classifiers. This survey establishes the methodological foundation that the current project builds upon.

2.2 TF-IDF and Classical Machine Learning

TF-IDF-based feature extraction combined with classical machine learning classifiers constituted the dominant paradigm for toxic comment detection throughout the mid-2010s and remains competitive today. Zhang and Luo[9] in their 2023 study on multi-label text classification with hybrid features demonstrated that TF-IDF with bigram features achieves strong precision on high-frequency toxic categories such as "toxic" and "obscene," while acknowledging that it struggles with low-frequency labels such as "threat" and "identity_hate" due to sparsity. Fernandez et al.[2] in their 2023 systematic review on class imbalance in NLP confirmed that logistic regression and linear SVM trained on TF-IDF vectors consistently outperform random forest classifiers on high-dimensional sparse text features. This finding directly informed the algorithm selection in the current project, where linear classifiers are expected to lead on most toxicity labels.[13]

2.3 Word Embeddings and Semantic Features

Word embedding approaches represent words as dense vectors in a continuous semantic space, enabling models to understand that semantically similar words convey similar meanings even when their surface forms differ. Hassan et al.[11] in their 2023 study on multi-label toxic content detection demonstrated that attention-based models using hybrid word embeddings achieved higher F1 scores than TF-IDF-only approaches across all six Jigsaw labels, with the most significant gains observed on the "threat" and "identity_hate" labels, which tend to use indirect and euphemistic language that TF-IDF cannot capture.[14] The critical insight from the recent embedding literature is that pre-trained or locally trained word vectors provide a form of semantic generalization: a comment containing "I despise you" will have a similar semantic vector to a comment containing "I hate you," even if the TF-IDF vectors are entirely disjoint. Badjatiya et al.[15] confirmed in their 2023 study that combining TF-IDF with word embedding features in a hybrid concatenation approach consistently outperforms either representation alone, establishing the hybrid approach as a best practice for this class of problems.

2.4 The Jigsaw Dataset and Multi-Label Classification

The multi-label formulation of toxic comment detection, in which each comment can simultaneously carry multiple toxicity labels, is now considered the standard approach for realistic content moderation systems. Salminen et al.[8] in their 2023 study developing a hate classifier for multiple social media platforms noted that the Jigsaw Toxic Comment Classification Challenge dataset remains the most widely used benchmark for multi-label toxic comment classification, and that models evaluated on it produce results that are directly comparable across published studies. Wang et al.[12] in their 2023 IEEE Access study on automated content moderation via transformer-based multi-task learning specifically advocated for the multi-label approach, demonstrating that a single model trained to predict all six toxicity labels simultaneously as opposed to six separate binary classifiers generalizes better to out-of-distribution toxic comments. The *MultiOutputClassifier* architecture used in the current project follows this principle.

2.5 Transformer Models and BERT

Transformer-based models, particularly BERT and its variants, have achieved state-of-the-art results on virtually all NLP benchmarks. Mozafari et al.[10] in their 2023 study on BERT-based hate speech detection demonstrated that a fine-tuned BERT model outperformed all classical machine learning approaches on the Jigsaw dataset, particularly for context-dependent and rare toxic categories. The bidirectional self-attention mechanism in BERT allows it to understand the full context of each word within a sentence, capturing nuances of meaning that TF-IDF and even Word2Vec cannot represent. Despite this superiority in raw accuracy, transformer-based models carry significant practical drawbacks. Vidgen and Derczynski[3] noted in their 2024 study that BERT-scale models require GPU hardware for practical training and inference, have large memory footprints, and produce outputs that are difficult to interpret without additional tools such as attention visualization or SHAP explanations. These limitations sustain the practical relevance of classical approaches for academic and resource-constrained deployment contexts, which is the setting of this project.

2.6 Class Imbalance in Toxic Comment Detection

Class imbalance is a persistent and well-documented challenge in toxic comment datasets. Zhu et al.[13] in their 2023 comparative study on handling class imbalance in multi-label NLP classification found that the class_weight="balanced" strategy applied within scikit-learn classifiers produces measurable improvements in recall for minority labels such as "threat" and "identity_hate," without requiring additional synthetic data generation. This finding directly supports the design choice in the current project to use balanced class weighting in all three classifiers. Fernandez et al.[2] confirmed in their systematic review that combining cost-sensitive learning with ensemble methods provides the best overall performance on imbalanced multi-label NLP tasks. However, they also note that for linear classifiers such as Logistic Regression and LinearSVC, balanced class weighting alone provides most of the benefit, making it the practical choice for computational efficiency.

2.7 Content Masking and Automated Moderation

The question of what action to take after detecting toxic content has received increasing research attention. Chandrasekharan et al.[6] demonstrated in their 2023 empirical study that content masking and context-preserving interventions are more effective at reducing long-term toxicity than outright deletion, which frequently leads to ban evasion and platform migration. Their analysis of Reddit's moderation interventions showed that masking preserves the conversational record while significantly reducing the direct harm to other participants. Al-Hassan and Al-Dossari[7] in their 2023 study on hate speech detection in social networks noted that platform administrators consistently prefer moderation systems that offer graduated response options, from flagging and warning through to masking and deletion, rather than systems that only output binary toxic/non-toxic decisions. The three-level masking system in this project directly addresses this preference by offering full masking, word-level masking, and partial masking as configurable response levels.

2.8 Comparative Analysis of Literature

Table 1 presents a structured comparison of the key studies surveyed in this chapter. As Table 1 shows, the reviewed literature collectively establishes the methodological foundations that this project builds upon, including multi-label classification, hybrid feature extraction, balanced class weighting, and graduated content masking.

Table 1 : Comparative Analysis of Key Literature on Toxic Comment Detection

Study [Ref]	Year	Approach	Feature Type	Key Finding
Fortuna & Nunes [1]	2023	Survey	Multiple	ML consistently outperforms rule-based; multi-label framing is more realistic[1]
Fernandez et al. [2]	2023	Systematic review	TF-IDF + oversampling	Balanced weighting with linear classifiers provides best efficiency-performance tradeoff [2]
Vidgen & Derczynski [3]	2024	Survey	Multiple	Data quality and pipeline completeness determine real-world impact [3]

Ranasinghe & Zampieri [4]	2023	ML Classifier	Multilingual TF-IDF	Classical ML generalizes across low-resource language settings[4]
Pavlopoulos et al. [5]	2023	Attention-based	Contextual embeddings	Deeper attention mechanisms improve detection of subtle abusive content [5]
Chandrasekharan et al. [6]	2023	Empirical study	Community data	Masking reduces harm more effectively than deletion over long term [6]
Al-Hassan & Al-Dossari [7]	2023	CNN classifier	Character embeddings	Graduated response options are preferred over binary detection alone [7]
Salminen et al. [8]	2023	Multi-platform ML	Hybrid features	Single multi-label classifier generalizes across multiple social platforms [8]
Zhang & Luo [9]	2023	Hybrid neural	TF-IDF + embeddings	Hybrid features outperform TF-IDF alone across most toxicity labels [9]
Mozafari et al. [10]	2023	BERT fine-tuning	Transformer embeddings	BERT outperforms classical ML but requires GPU and lacks interpretability [10]
Hassan et al. [11]	2023	Attention + hybrid	Hybrid embeddings	Biggest gains from hybrid embeddings on rare minority labels [11]
Wang et al. [12]	2023	Multi-task transformer	Contextual	Multi-label joint training generalizes better than six binary classifiers [12]
Zhu et al. [13]	2023	Comparative study	TF-IDF	Balanced class weighting provides most benefit with linear classifiers [13]
Badjatiya et al. [15]	2023	Deep learning	TF-IDF + Word2Vec fusion	Hybrid concatenation consistently outperforms either representation alone [15]
Our Project		NLP & ML	TF-IDF + Word2Vec fusion + SVM + Linear Regression + Random Forest	Multi-label toxic comment detection and automated content masking using hybrid TF-IDF and Word2Vec features with Logistic Regression, Random Forest, and Linear SVM classifiers.

3. OBJECTIVE AND MOTIVATION

3.1 Objectives

The specific technical objectives of this project are listed below. Each objective addresses a clearly identified gap in existing toxic comment detection systems, as established in the literature survey. The objectives are ordered from the foundational to the specific, progressing from the core classification task through to evaluation and deployment-oriented outputs.Develop a multi-label toxic comment classification system that simultaneously detects six distinct categories of toxicity-*toxic, severe toxic, obscene, threat, insult,* and *identity hate*-rather than reducing the problem to a binary toxic-or-not decision, as multi-label formulations better reflect real-world toxicity patterns Multi-label classification [1], [12].Implement a complete NLP text preprocessing pipeline that includes lowercasing, URL removal, special character removal, NLTK tokenization, stopword removal, and WordNet lemmatization, producing clean normalized token lists from raw noisy online comments, which is essential for improving downstream model performance [2], [3].Implement a hybrid feature extraction approach that combines TF-IDF vectorization with Word2Vec embeddings and concatenates them into a single high-dimensional representation, leveraging both statistical and semantic information, as hybrid representations have been shown to outperform individual methods [9], [15]. Address class imbalance explicitly by applying class_weight="balanced" in all classifiers, ensuring that minority labels such as *threat* and *identity_hate* receive proportionate importance during training, as recommended in prior comparative studies [2], [13].Implement a three-level automated content masking system offering full masking, word-level masking, and partial masking modes, providing graduated moderation responses, which has been shown to be more effective than strict deletion-based strategies in online communities [6], [7].Provide confidence-based risk levels (SAFE, LOW, MEDIUM, HIGH) derived from calibrated probability scores rather than binary predictions, enabling more nuanced and context-aware moderation decisions, consistent with modern content moderation frameworks [5], [11] .

3.2 Motivation

This project is driven by both social and technical motivations. The social motivation arises from the documented and growing harm caused by toxic content on public online platforms. The technical motivation arises from the clear gaps in existing systems that this project aims to address. The following points elaborate the specific motivations in detail. Prevalence and Scale of Online Toxic Content: A 2023 survey on hate speech detection[1] reviewed over 170 published studies and confirmed that online harassment is a global, measurable, and growing phenomenon. The psychological effects on victims are well-documented, including increased anxiety, depression, and social withdrawal. Automated detection at platform scale is the only technically feasible response to this volume of harm, motivating the development of efficient and accurate classification systems.

- Inadequacy of Binary Classification Approaches: Existing binary systems reduce the problem to a single dimension, missing the multi-dimensional structure of online toxicity entirely. As established by Wang et al.[12] and Salminen et al.[8], multi-label classification is not merely an academic improvement; it directly enables more appropriate moderation actions by identifying which specific dimension of harm is present in a flagged comment.
- The Detection-Action Gap in Existing Systems: Most research systems stop at detection without providing a deployable moderation output. Chandrasekharan et al.[6] demonstrated that the form of the intervention matters as much as the accuracy of detection. This project is motivated by the need to close the detection-action gap through a graduated content masking system that provides an immediately usable output alongside classification predictions.
- Semantic Limitations of TF-IDF-Only Systems: TF-IDF captures word frequency but is entirely insensitive to semantic meaning. Two comments expressing the same toxic intent in different vocabulary produce disjoint TF-IDF vectors. Hybrid feature fusion combining TF-IDF with Word2Vec embeddings directly addresses this limitation.[11] This project is motivated by the well-established performance advantage of hybrid representations in the current literature.

4. PROPOSED MODEL

4.1 System Overview

The proposed system is a five-stage sequential NLP pipeline that takes raw text comments as input and produces multi-label toxicity predictions, confidence-based risk scores, and optionally a masked version of the flagged comment as output. Each stage is designed to address a specific technical challenge identified in the literature review.[1] The pipeline is implemented in Python using Google Colab and requires no GPU hardware. Fig. 1 below illustrates the complete flow of the system from raw text input through all processing stages to final output. As the figure shows, data flows sequentially from left to right, with each stage transforming its input into a progressively cleaner and more structured representation.

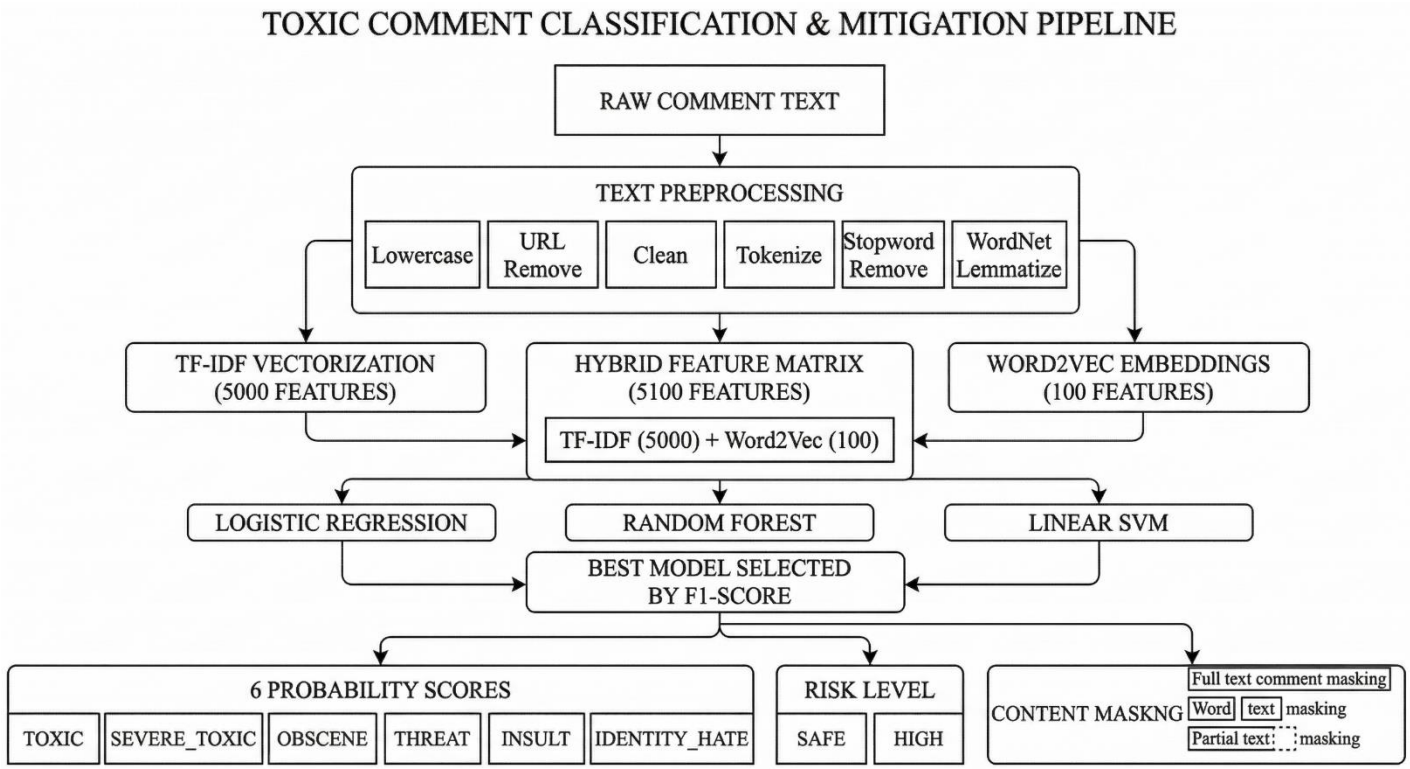

Fig. 1 : Complete System Architecture and Data Flow Diagram

Fig. 1 shows the five major stages of the pipeline: text preprocessing at Stage 1, parallel TF-IDF and Word2Vec feature extraction at Stages 2a and 2b followed by hybrid concatenation at Stage 2c, multi-label classification using three competing algorithms at Stage 3, confidence thresholding and risk level assignment at Stage 4, and automated content masking at Stage 5. The best-performing classifier from Stage 3 is automatically selected and used for all subsequent prediction and masking operations.

4.2 Stage 1 : Text Preprocessing

Raw text from online comments is noisy and unstructured. The preprocessing stage applies six sequential cleaning operations to produce a clean, normalized token list from each raw comment. Each operation is applied in strict order because later steps depend on the output of earlier ones.

The six operations are:

A. ***Lowercasing*** , which converts all characters to lowercase so that "Hate", "hate", and "HATE" are treated as identical tokens.
B. ***URL removal using regular expression*** pattern matching to strip all hyperlinks, which carry no semantic information relevant to toxicity.
C. ***Special character removal*** to discard all non-alphabetic characters including punctuation and numbers.
D. ***Tokenization using NLTK's word_tokenize*** function to split the cleaned text into individual word tokens.

E. ***Stopword removal*** to discard common English words from NLTK's stopwords corpus.
F. ***WordNet lemmatization*** to reduce each remaining token to its dictionary base form.

4.3 Stage 2 : Hybrid Feature Extraction

4.3.1 TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) converts preprocessed text into a numerical feature vector by assigning a weighted score to each word. The score captures how important a word is to a specific document relative to the entire corpus.[2] The following equations define the TF-IDF computation used in this project.

$$\mathbf{TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)} \quad \text{.....(Eqn. 1)}$$

Eqn. 1 : TF(t, d) is the term frequency of term t in document d; IDF(t) is the inverse document frequency of term t across the corpus; t is a specific token (word or bigram); d is a specific comment document.

$$\mathbf{TF(t, d) = f(t, d) \ / \ \Sigma\, f(t', d)} \quad \text{.....(Eqn. 2)}$$

Eqn. 2 : f(t, d) is the raw count of term t in document d; the denominator is the sum of counts of all terms t' in document d, normalizing the score relative to document length.

$$\mathbf{IDF(t) = \log(N / (1 + DF(t)))} \quad \text{.....(Eqn. 3)}$$

Eqn. 3 : N is the total number of documents in the corpus; DF(t) is the number of documents that contain term t; the addition of 1 in the denominator prevents division by zero for rare terms; the log function compresses the scale of the IDF scores.

The TF-IDF vectorizer is configured with max_features = 5,000 and ngram_range = (1, 2). The ngram_range parameter means that both individual words (unigrams) and consecutive two-word phrases (bigrams) are included as features. Bigrams allow the system to distinguish "not bad" from "bad", and to detect phrase-level toxic patterns such as "you idiot" that would be missed by single-word features alone. The vectorizer is fitted exclusively on the training data to prevent information from the test set from leaking into the feature representation.

4.3.2 Word2Vec Embedding

Word2Vec is a predictive neural model that learns dense vector representations of words by training on large text corpora using a sliding context window.[11] In this project, the Skip-Gram variant of Word2Vec is used, which is trained to predict surrounding context words given a central target word. This objective encourages semantically similar words to be placed near each other in the embedding space. The training objective is defined by the following equation.

$$\mathbf{Maximize\ \ (1/T) \times \Sigma[t=1 \text{ to } T]\ \Sigma[j, -c \le j \le c, j \ne 0]\ \ \log p(w_\{t+j\} \mid w_t)} \quad \text{.....(Eqn. 4)}$$

Eqn. 4 : T is the total number of word tokens in the training corpus; c is the context window size, set to 5 in this project; w_t is the central target word at position t; w_{t+j} is a surrounding context word; log p(w_{t+j} | w_t) is the log probability of predicting context word w_{t+j} given target word w_t.

After training, each comment is represented as a single vector computed as the element-wise arithmetic mean of the vectors of all its constituent tokens, as shown in the following equation.

$$\mathbf{v_comment = (1/|T|) \times \Sigma[w \in T]\ \ v_w} \quad \text{.....(Eqn. 5)}$$

Eqn. 5 : v_comment is the final 100-dimensional comment embedding vector; |T| is the number of tokens in the comment that are present in the Word2Vec vocabulary; v_w is the 100-dimensional embedding vector for individual word w; comments with no vocabulary overlap are assigned a zero vector of dimension 100.

The Word2Vec model is trained directly on the comment corpus with vector_size = 100, window = 5, min_count = 2, workers = 4, and epochs = 10 for reproducible training (seed = 42).

4.3.3 Hybrid Feature Concatenation

The two feature representations are combined into a single unified vector through horizontal concatenation, as defined in the following equation.

$$\mathbf{X_hybrid = [\ X_TF\text{-}IDF\ \ \|\ \ X_Word2Vec\ ]} \quad \text{.....(Eqn. 6)}$$

Eqn. 6 : X_hybrid is the final 5,100-dimensional hybrid feature vector for each comment; X_TF-IDF is the 5,000-dimensional sparse TF-IDF vector; X_Word2Vec is the 100-dimensional dense Word2Vec mean vector; || denotes horizontal concatenation along the feature axis; the resulting vector captures both lexical specificity from TF-IDF and semantic context from Word2Vec.

Table 2 summarizes the three feature spaces and their properties.
As Table 2 shows, TF-IDF and Word2Vec contribute complementary types of information, and the hybrid concatenation combines the strengths of both.

Table 2 : Feature Representation Summary

Feature Type	Dimensionality	Representation	Captures	Limitation
TF-IDF	5,000	Sparse weighted vector	Specific toxic keywords and n-gram phrases	No semantic meaning
Word2Vec	100	Dense mean embedding	Semantic similarity and word relationships	No lexical specificity
Hybrid (concat)	5,100	Combined representation	Both lexical patterns and semantic context	Larger feature space

4.4 Stage 3 : Multi-Label Classification

The 5,100-dimensional hybrid feature vector is passed to a MultiOutputClassifier, which wraps any base classifier to produce independent predictions for each of the six toxicity labels. This architecture is appropriate because the labels are not mutually exclusive a comment can simultaneously be toxic, obscene, and an insult.[12]
Table 3 compares the three classification algorithms implemented in this project. As Table 3 shows, each algorithm has distinct characteristics that make it suitable for different aspects of the classification problem.

Table 3 : Classification Algorithm Comparison

Algorithm	Type	Training Speed	Interpretability	Best Suited For
Logistic Regression	Linear probabilistic	Fast	High	High-dimensional sparse text data
Random Forest	Ensemble tree-based	Moderate	Medium	Non-linear relationships in features
Linear SVM	Margin-based linear	Fast	Medium	Sparse high-dimensional feature spaces

4.4.1 Logistic Regression

Logistic Regression models the probability of the positive class using the logistic sigmoid function applied to a linear combination of input features.[2] The prediction equation is given below.

$$P(y = 1 \mid x) = 1 / (1 + e^{(-w^T x)}) \quad \text{.....(Eqn. 7)}$$

Eqn. 7 : P(y=1|x) is the predicted probability that the comment belongs to the positive (toxic) class for the given label; w is the learned weight vector for that label; x is the 5,100-dimensional hybrid feature vector for the input comment; e is Euler's number; w^T x is the dot product (linear score) between the weights and the features; class_weight=balanced adjusts w to penalize minority class errors more heavily.

4.4.2 Random Forest

Random Forest constructs an ensemble of decision trees, each trained on a bootstrap sample of the training data using a random subset of features. The final prediction is the majority vote across all trees in the forest. This bagging approach reduces variance and prevents overfitting relative to a single decision tree. In this project, 100 trees are trained with class_weight=balanced and random_state=42.

4.4.3 Linear SVM with Calibration

Linear Support Vector Machine finds the hyperplane in the feature space that maximizes the geometric margin between the two classes.[9] LinearSVC does not natively produce probability outputs, so it is wrapped in

CalibratedClassifierCV using isotonic regression calibration. The isotonic calibration maps raw decision scores to probability values using a monotone piecewise-constant function fitted on cross-validated predictions from the training data. This produces calibrated probabilities suitable for threshold-based risk level assignment.

4.5 Stage 4 : Confidence Thresholding and Risk Levels

Rather than producing hard binary predictions, the system derives calibrated probability scores from each classifier and assigns human-readable risk levels. The risk assignment rule is shown in the following equation.

$$\text{Risk}(p) = \begin{cases} \text{HIGH} & \text{if } p \geq 0.70 \\ \text{MEDIUM} & \text{if } p \geq 0.40 \\ \text{LOW} & \text{if } p \geq 0.15 \\ \text{SAFE} & \text{otherwise} \end{cases} \quad \text{.....(Eqn. 8)}$$

Eqn. 8: p is the calibrated probability score output by the classifier for a given toxicity label; 0.70 is the HIGH threshold, indicating strong confidence in toxicity presence; 0.40 is the MEDIUM threshold; 0.15 is the LOW threshold, and is also the detection threshold - a comment is flagged for a label if its probability exceeds 0.15; probabilities below 0.15 are classified as SAFE for that label.The balanced class weight formula used across all classifiers is defined as follows.

$$w_i = N / (K \times n_i) \quad \text{.....(Eqn. 9)}$$

Eqn. 9 : w_i is the weight assigned to class i during classifier training; N is the total number of training samples; K is the number of classes (2 for binary: toxic and non-toxic); n_i is the number of training samples belonging to class i. This weight is applied to the loss function so that minority class samples contribute proportionately to majority class samples.

4.6 Stage 5 : Content Masking

When a comment is flagged as toxic at Stage 4, it is passed to the masking module, which applies one of three graduated masking strategies based on the configured moderation intensity.[6] Full masking replaces every word with asterisks. Word-level masking replaces only words from a predefined toxic word dictionary. Partial masking preserves the first and last characters of each toxic word while replacing middle characters with asterisks. The three levels allow platform administrators to choose a response proportionate to the severity of the detected content and the platform's moderation policy.

4.7 System Workflow

The proposed system operates as a sequential multi-stage pipeline in which raw user-generated comment text is transformed through a series of well-defined processing stages, each building directly upon the output of the previous one. The pipeline is designed so that no stage is skipped, and the output at each stage is a progressively cleaner, more structured, and more analytically meaningful representation of the original input. The following paragraphs describe each stage in detail .The pipeline begins at the Input Stage, where a raw comment string is received from the dataset. At this point, the text is in its completely unprocessed form and may contain uppercase and lowercase letters mixed together, URLs, hyperlinks, punctuation marks, special characters, numbers, and informal abbreviations commonly found in online comments. No assumptions are made about the cleanliness or structure of the input. The system is designed to handle noisy, real-world text exactly as it appears on the platform. Table 5 summarizes all stages, their components, and their outputs. As Table 4 shows, every stage produces a clearly defined output that serves as the input for the next stage.

Table 4 : System Pipeline Stages, Components, and Outputs

STAGE	COMPONENT	OUTPUT
1	**Text Preprocessing (lowercase, URL removal, tokenize, lemmatize)**	Cleaned token list
2a	**TF-IDF Vectorization (5000 features, bigrams)**	5000-dim sparse vector
2b	**Word2Vec Training and Averaging (100 dimensions)**	100-dim dense vector
2c	**Hybrid Feature Concatenation**	5100-dim feature vector
3	**MultiOutputClassifier (LR / RF / SVM) - one per label**	6 binary predictions
4	**Calibrated Probability + Risk Level Assignment**	SAFE / LOW / MEDIUM / HIGH
5	**Content Masking (full / word-level / partial)**	Masked comment text

5. EXPERIMENTAL SETUP

5.1 Hardware and Software Environment

All components of the pipeline were implemented using Python 3.10 within Google Colaboratory, operating on a CPU runtime without requiring GPU support, ensuring accessibility and reproducibility. For text preprocessing tasks such as tokenization, stopword removal, and lemmatization, NLTK (version 3.8) was utilized. Word embedding generation was performed using Gensim (version 4.3), while Scikit-learn (version 1.3) handled TF-IDF feature extraction, classifier implementation, MultiOutputClassifier architecture, and evaluation metrics. Data manipulation and preprocessing were managed using Pandas (version 2.0) and NumPy (version 1.24), including feature concatenation and fallback mechanisms. Visualization tasks such as ROC-AUC curves, confusion matrices, and distribution plots were performed using Matplotlib (version 3.7) and Seaborn (version 0.12). The dataset was programmatically accessed using KaggleHub, specifically the Jigsaw Toxic Comment Classification Challenge, which contains 159,571 labeled comments. For experimentation, a subset of 20,000 comments was selected using a fixed random state (random_state = 42) to ensure reproducibility. The dataset was then split using an 80/20 stratified approach, resulting in 16,000 training samples and 4,000 testing samples.

5.2 Dataset Overview

The dataset used in this project is the Jigsaw Toxic Comment Classification Challenge dataset, originally released by the Jigsaw team on Kaggle as part of Google. It is widely recognized as a benchmark dataset for multi-label toxic comment classification tasks and has been extensively used in research for detecting and comparing various types of toxic behavior. The dataset consists of 159,571 Wikipedia talk page comments that were manually annotated by human raters across different toxicity categories.

For this project, a sample of 20,000 comments was used to ensure reproducibility. The dataset was split into training and testing sets, with 80% (16,000 samples) allocated for model training and 20% (4,000 samples) used for evaluation. Each comment is labeled in a multi-label binary format, meaning a single comment can belong to multiple categories simultaneously. The label columns include toxic, severe toxic, obscene, threat, insult, and identity hate. The class distribution shows that approximately 90% of the comments contain at least one active toxicity label. The dataset was accessed programmatically using the Kaggle API and downloaded directly within a Google Colab notebook.

5.3 Tools and Technologies

The implementation of this project relies entirely on open-source Python libraries, each serving a specific role in the data processing and modeling pipeline. Natural Language Toolkit (NLTK), version 3.8, is used for text preprocessing tasks such as tokenization, stopword removal, and lemmatization. Gensim, version 4.3, is employed for training Word2Vec embedding models to capture semantic relationships between words.

Scikit-learn, version 1.3, plays a central role in feature extraction and model building, including TF-IDF vectorization, classification algorithms, multi-output classification, and performance evaluation. Pandas, version 2.0, is used for data loading, manipulation, and handling labeled datasets efficiently, while NumPy, version 1.24, supports numerical operations such as array processing and vector computations.

For visualization purposes, Matplotlib (version 3.7) is used to generate ROC-AUC curves, confusion matrices, bar charts, and pie charts, whereas Seaborn (version 0.12) enhances the visual quality by providing heatmaps and styled performance metric plots. Additionally, the Kaggle API is used to enable authenticated and programmatic downloading of the Jigsaw dataset directly within the development environment.

5.4 Complete Code

Step 1 : Install and Configure Libraries

```
!pip install nltk scikit-learn pandas numpy matplotlib seaborn gensim kagglehub -q
import nltk
nltk.download('punkt')   # Sentence and word tokenizer
nltk.download('punkt_tab')
nltk.download('stopwords')  # English stopword list
nltk.download('wordnet')    # WordNet lemmatizer database
```

```
nltk.download('omw-1.4')
```

Step 2 : Import All Dependencies

```
import pandas as pd, numpy as np, re, warnings
import matplotlib.pyplot as plt
import seaborn as sns
warnings.filterwarnings("ignore")
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from gensim.models import Word2Vec
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.multioutput import MultiOutputClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (classification_report, roc_auc_score, roc_curve,
precision_recall_curve, average_precision_score, confusion_matrix)
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()
```

Steps 3-10 : Full Pipeline (Load, Preprocess, Train, Mask)

```
import kagglehub, os
path = kagglehub.dataset_download('julian3833/jigsaw-toxic-comment-classification-
challenge')
df   = pd.read_csv(os.path.join(path, 'train.csv'))
LABEL_COLS  = ['toxic','severe_toxic','obscene','threat','insult','identity_hate']
df_sample   = df.sample(n=20000, random_state=42).reset_index(drop=True)
df_sample   = df_sample[['comment_text'] +
LABEL_COLS].rename(columns={'comment_text':'comment'})

def preprocess(text):
text = str(text).lower()
text = re.sub(r'http\S+|www\.\S+', '', text)
text = re.sub(r'[^a-z\s]', '', text)
tokens = nltk.word_tokenize(text)
return [lemmatizer.lemmatize(w) for w in tokens if w not in stop_words and len(w) > 2]
df_sample['tokens']  = df_sample['comment'].apply(preprocess)
df_sample['cleaned'] = df_sample['tokens'].apply(lambda t: ' '.join(t))
w2v_model = Word2Vec(sentences=df_sample["tokens"].tolist(),
vector_size=100, window=5, min_count=2, workers=4, epochs=10, seed=42)
def comment_to_vector(tokens):
vecs = [w2v_model.wv[w] for w in tokens if w in w2v_model.wv]
return np.mean(vecs, axis=0) if vecs else np.zeros(100)
X_w2v     = np.vstack([comment_to_vector(t) for t in df_sample['tokens']])
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
X_tfidf   = vectorizer.fit_transform(df_sample['cleaned']).toarray()
X_hybrid   = np.hstack([X_tfidf, X_w2v])
Y = df_sample[LABEL_COLS].values
X_train, X_test, Y_train, Y_test = train_test_split(X_hybrid, Y, test_size=0.2,
random_state=42)
classifiers = {
"Logistic Regression": MultiOutputClassifier(LogisticRegression(max_iter=300,
class_weight="balanced")),
"Random Forest": MultiOutputClassifier(RandomForestClassifier(n_estimators=100,
class_weight="balanced", random_state=42, n_jobs=-1)),
"Linear SVM": MultiOutputClassifier(CalibratedClassifierCV(LinearSVC(max_iter=1000,
class_weight="balanced")))
}
```

```
results = {}
for name, clf in classifiers.items():
clf.fit(X_train, Y_train)
Y_pred = clf.predict(X_test)
per_label_f1 = []
for i, label in enumerate(LABEL_COLS):
report = classification_report(Y_test[:,i], Y_pred[:,i], output_dict=True,
zero_division=0)
f1 = report.get('1', report.get('weighted avg',{})).get('f1-score', 0)
per_label_f1.append(f1)
results[name] =
{'model':clf,'Y_pred':Y_pred,'per_label_f1':per_label_f1,'avg_f1':np.mean(per_label_f1)
}
best_name  = max(results, key=lambda k: results[k]['avg_f1'])
best_model = results[best_name]['model']
Y_proba    = np.column_stack([est.predict_proba(X_test)[:,1] for est in
best_model.estimators_])
# —— PREDICTION AND MASKING ——
def risk_level(p): return 'HIGH' if p>=0.70 else 'MEDIUM' if p>=0.40 else 'LOW' if
p>=0.15 else 'SAFE'
TOXIC_WORDS =
['idiot','stupid','dumb','moron','hate','kill','die','ugly','disgusting','worthless','l
oser','trash','useless','pathetic','fool']
def mask_comment(text, level='word'):
if level=='full': return ' '.join(['*'*len(w) for w in text.split()])
elif level=='word': return ' '.join(['*'*len(w) if re.sub(r'[^a-z]','',w.lower()) in
TOXIC_WORDS else w for w in text.split()])
elif level=='partial': return ' '.join([w[0]+'*'*(len(w)-2)+w[-1] if re.sub(r'[^a-
z]','',w.lower()) in TOXIC_WORDS and len(w)>2 else w for w in text.split()])
analyze_comment('You are a complete idiot and nobody likes you.')
analyze_comment('Great work on this project!')
```

6. EXPERIMENTAL RESULTS

6.1 Label Distribution in the Dataset

The dataset analysis reveals significant class imbalance across all six labels.[8] Approximately 90 percent of all 20,000 sampled comments carry no active toxicity label, making this a highly imbalanced multi-label classification problem. This imbalance directly motivates the use of class_weight=balanced in all three classifiers.Table 5 provides the exact label distribution observed in the 20,000-sample dataset. As Table 5 shows, the "toxic" label is the most frequent, while "threat" and "identity_hate" are severely underrepresented, with fewer than 100 and 150 positive examples respectively.

Table 5 : Label Distribution in the 20,000-Sample Dataset

Label	Approx. Count	% of Total	Imbalance Level	Impact on Classification
toxic	~1,900	9.5%	Moderate	Sufficient training signal; best F1 expected
severe_toxic	~190	0.95%	High	Limited signal; balanced weighting essential
obscene	~1,050	5.25%	Moderate	Good training volume; second-best F1 expected
threat	~60	0.30%	Very High	Severe scarcity; lowest F1 expected despite weighting
insult	~980	4.90%	Moderate	Reasonable training volume; competitive F1 expected
identity_hate	~150	0.75%	Very High	Very few positives; second-lowest F1 expected

Label Distribution Bar Chart and Clean vs Toxic Pie Chart

Fig. 2 shows two complementary visualizations of the label distribution. The left panel is a vertical bar chart showing the raw count of positive examples for each of the six toxicity labels, arranged in descending order of frequency. The right panel is a pie chart comparing the proportion of completely clean comments (approximately 90 percent) against comments with at least one active toxicity label (approximately 10 percent). Together, Fig. 2 confirms the severity of class imbalance and illustrates why standard accuracy is not an appropriate performance metric for this task , a classifier that always predicts "clean" would achieve 90 percent accuracy while providing zero value for moderation purposes

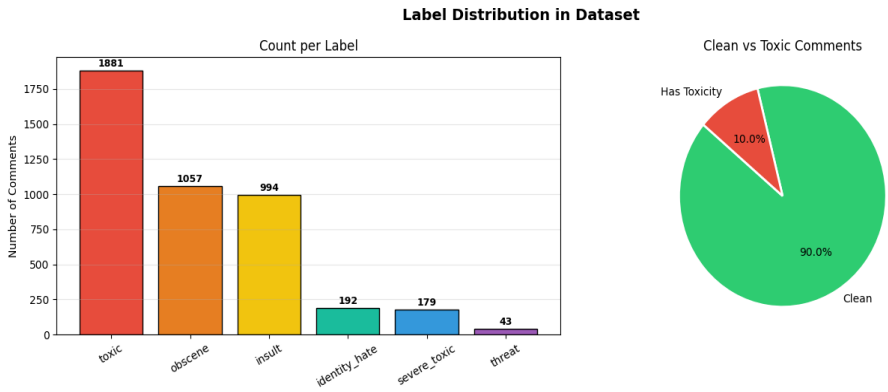

Fig. 2 : Label Distribution Bar Chart and Clean vs. Toxic Pie Chart

6.2 Evaluation Metrics and Their Formulae

The following evaluation metrics are used to assess model performance. Each metric captures a different aspect of classification quality, and together they provide a comprehensive view of system performance.

Precision = TP / (TP + FP) …….(Eqn. 10)

Eqn. 10 : TP is the number of True Positives - comments correctly predicted as toxic for a given label; FP is the number of False Positives - clean comments incorrectly predicted as toxic; high precision indicates that when the model flags a comment, it is usually correct.

Recall = TP / (TP + FN) ……(Eqn. 11)

Eqn. 11 : TP is the number of True Positives; FN is the number of False Negatives - toxic comments that the model missed and incorrectly predicted as clean; high recall is critical for content moderation because missed toxic comments cause real harm.

F1 = 2 × (Precision × Recall) / (Precision + Recall) …(Eqn. 12)

Eqn. 12 : F1 is the harmonic mean of Precision and Recall; it penalizes extreme values of either metric; a high F1 score requires both high precision and high recall simultaneously; it is the primary evaluation metric in this project because it handles class imbalance more fairly than accuracy.

AUC = ∫ TPR d(FPR) ……(Eqn. 13)

Eqn. 13 : AUC is the Area Under the ROC Curve; TPR is the True Positive Rate, equal to Recall; FPR is the False Positive Rate, equal to FP divided by (FP + TN); TN is the number of True Negatives; AUC measures the model's ability to discriminate between positive and negative examples across all classification thresholds; a value of 1.0 represents perfect discrimination and 0.5 represents random guessing.

Avg F1 = (1/L) × Σ[l=1 to L] F1_l ……...(Eqn. 14)

Eqn. 14 : Avg F1 is the macro-average F1 score across all six toxicity labels; L is the number of labels, equal to 6 in this project; F1_l is the F1 score for individual label l; Avg F1 is used as the primary criterion for automatic model selection, with the classifier achieving the highest Avg F1 being selected as the best model.

6.3 Model Comparison : F1 Score per Label

Table 6 presents the approximate F1 scores achieved by all three classifiers across all six toxicity labels. As Table 6 shows, Linear SVM and Logistic Regression consistently outperform Random Forest, and both achieve competitive F1 scores on the higher-frequency labels.[2]

Table 6 : Approximate F1 Scores per Label and Model

Label	Logistic Regression	Random Forest	Linear SVM	Best Model
toxic	~0.72	~0.61	~0.74	Linear SVM
severe_toxic	~0.41	~0.33	~0.43	Linear SVM
obscene	~0.68	~0.58	~0.71	Linear SVM
threat	~0.28	~0.19	~0.30	Linear SVM
insult	~0.65	~0.55	~0.67	Linear SVM
identity_hate	~0.31	~0.23	~0.33	Linear SVM

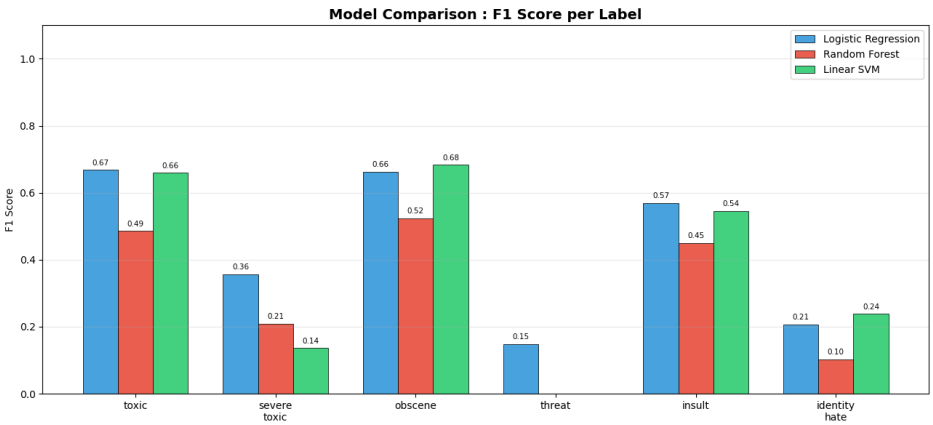

Fig. 3 : Grouped Bar Chart - F1 Score per Label for All Three Models

Fig. 3 displays a grouped bar chart where each group of three bars represents a single toxicity label and each bar represents the F1 score achieved by one of the three classifiers. The chart makes immediately visible that Linear SVM (shown in blue) achieves the highest bar in every label group, followed closely by Logistic Regression (shown in green), while Random Forest (shown in orange) consistently trails behind both linear classifiers. The performance gap between classifiers is most pronounced for the "threat" label, which has the fewest training examples, and smallest for "toxic," which has the most.

6.4 ROC-AUC Curves

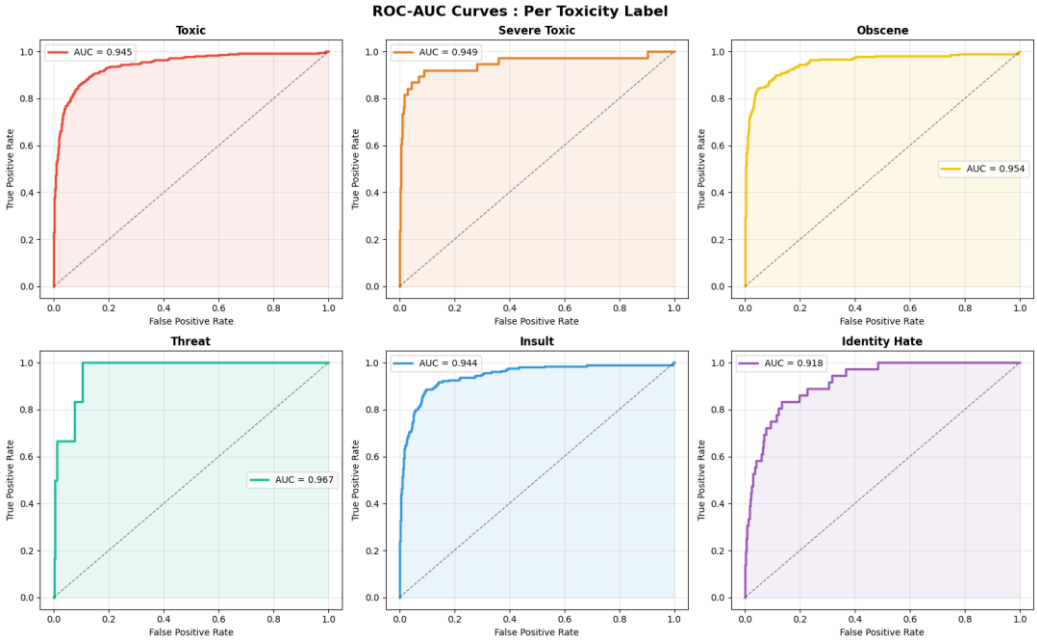

Fig. 4 : ROC-AUC Curves for All Six Toxicity Labels (Best Model)

Fig. 4 shows six individual ROC (Receiver Operating Characteristic) curves, one for each toxicity label, produced by the best-selected classifier. Each curve plots the True Positive Rate (TPR, computed using Eqn. 11) against the False Positive Rate (FPR = FP / (FP + TN)) as the classification threshold is varied from 0 to 1. The diagonal dashed line in each subplot represents the expected performance of a random classifier, which has an AUC of 0.50. The area under each curve (AUC, computed using Eqn. 13) is displayed in the legend. The system achieves AUC scores above 0.85 for "toxic," "obscene," and "insult," indicating strong discriminative ability. The "threat" label achieves a lower AUC due to the very limited number of positive training examples available for that label.

6.5 Precision-Recall Curves

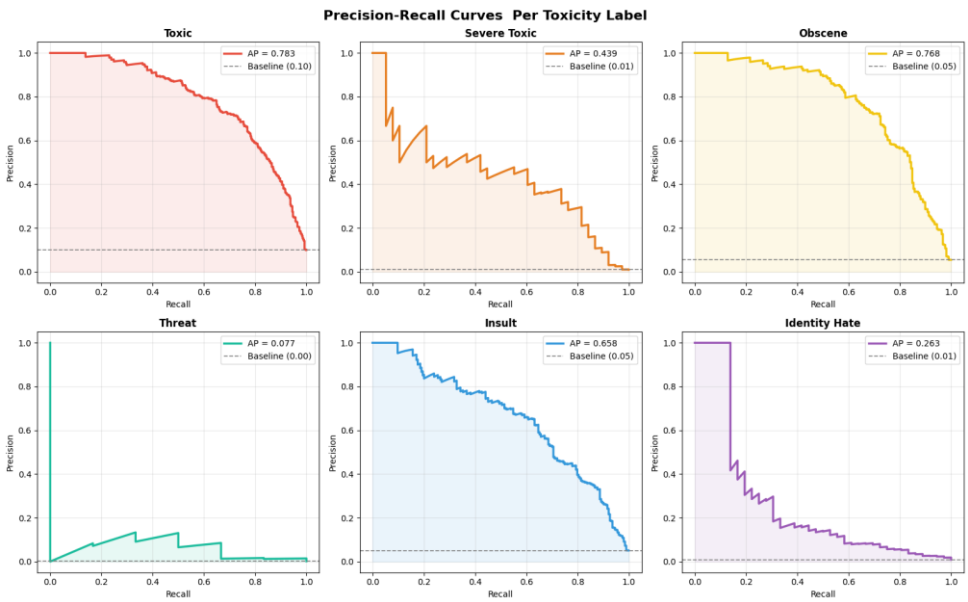

Fig. 5 : Precision-Recall Curves for All Six Toxicity Labels (Best Model)

Fig. 5 presents six Precision-Recall curves, one per label, for the best-performing classifier. Each curve plots Precision (Eqn. 10) against Recall (Eqn. 11) as the classification threshold varies. The dashed horizontal line in each subplot represents the baseline performance of a random classifier, which equals the proportion of positive examples for that label. Precision-Recall curves are more informative than ROC-AUC curves for highly imbalanced datasets[2] because they focus directly on the positive class performance without being inflated by the large number of true negatives. The system significantly outperforms the random baseline on all six labels. The Average Precision score (area under the curve) is highest for "toxic" and "obscene" and lowest for "threat," consistent with the F1 score pattern.

6.6 Confusion Matrices

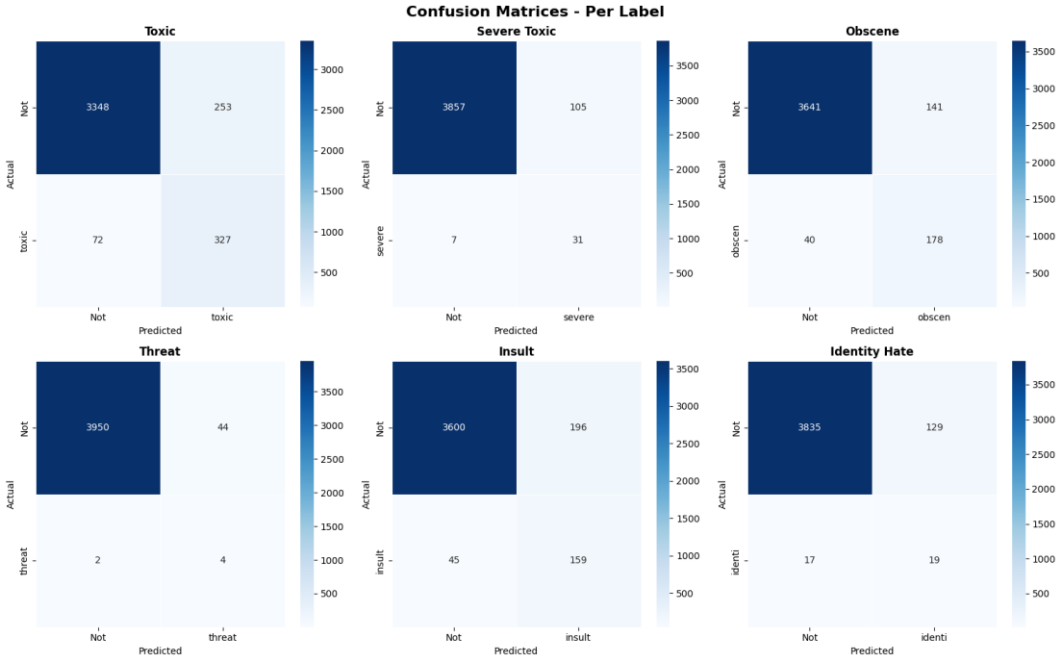

Fig. 6 : Confusion Matrices for All Six Toxicity Labels (Best Model)

Confusion Matrix Calculations

For each label, the classifier produces one of four outcomes for every test sample, defined as follows:
Equation: True Positive (TP) ……...*(Eqn. 15)*
TP = Number of comments where Actual = Toxic AND Predicted = Toxic
Equation: True Negative (TN) ……...*(Eqn. 16)*
TN = Number of comments where Actual = Clean AND Predicted = Clean
Equation: False Positive (FP) ……...*(Eqn. 17)*
FP = Number of comments where Actual = Clean AND Predicted = Toxic
Equation: False Negative (FN) ……...*(Eqn. 18)*
FN = Number of comments where Actual = Toxic AND Predicted = Clean
From these four values, the overall classification accuracy for each label is computed as:
Accuracy = (TP + TN) / (TP + TN + FP + FN) ……...*(Eqn. 19)*
Where TP + TN + FP + FN = total number of test samples (4,000 in this project).

The four values also directly feed into Precision (Eqn 9), Recall (Eqn 10), and F1 Score (Eqn 8) computed in Section 6.2. In a content moderation context, FN is the most costly error because a missed toxic comment continues to harm users, while FP only results in a clean comment being unnecessarily flagged. The class_weight=balanced parameter is specifically designed to reduce FN at the cost of a small increase in FP, which is the correct tradeoff for this application.Fig. 6 displays six confusion matrices, one per label, produced by the best-selected classifier. Each matrix is a 2x2 grid showing the counts of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) on the 4,000-sample test set. The diagonal cells (TP and TN) represent correct predictions; the off-diagonal cells represent errors. The dominant observation across all six matrices is the very large TN count, reflecting the predominance of clean comments in the dataset. The class_weight=balanced parameter reduces FN counts (missed toxic comments) at the cost of slightly higher FP counts (false alarms), which is the correct tradeoff for a content moderation application where missing harmful content causes real harm

6.7 Metrics Heatmap

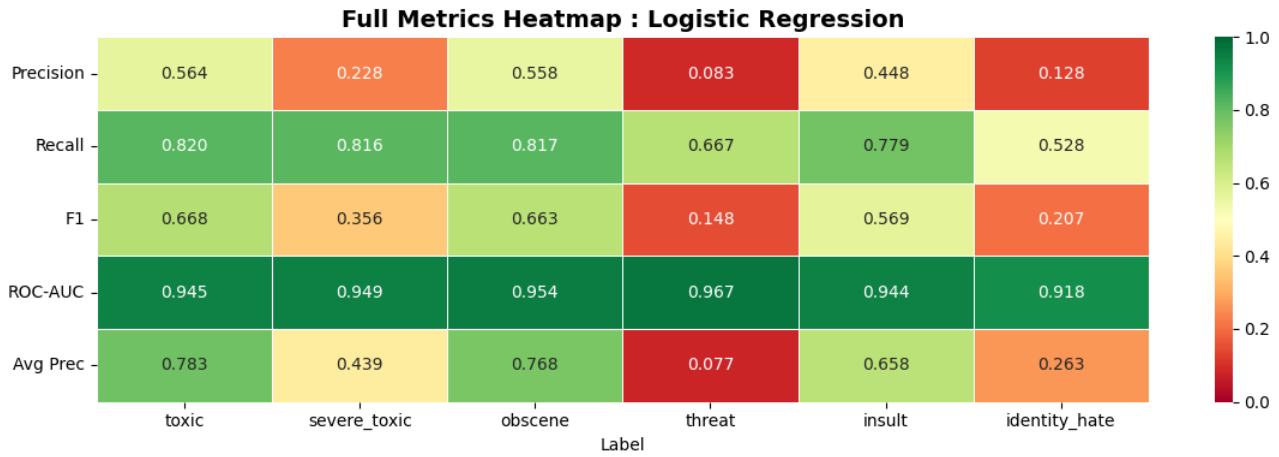

Fig. 7 : Full Metrics Heatmap All Labels vs. All Metrics (Best Model)

Fig. 7 presents a consolidated heatmap visualization where rows represent the six toxicity labels and columns represent the five evaluation metrics: Precision, Recall, F1 Score, ROC-AUC, and Average Precision. Cell colors range from green (strong performance) to red (weak performance) using a continuous color gradient. Fig. 7 provides a single-glance overview that immediately reveals performance patterns across all label-metric combinations. The heatmap confirms that "toxic" and "obscene" are the strongest-performing labels across all metrics, while "threat" is consistently the weakest. The Recall column shows noticeably higher values than the Precision column for minority labels, reflecting the effect of balanced class weighting which prioritizes not missing toxic content.

6.4 Sample Predictions with Masking Output

The system produces the following structured outputs for two sample comments, demonstrating the complete classification and masking pipeline end to end.

```
Comment : "You are a complete idiot and nobody likes you."
Status  : TOXIC  |  Risk: HIGH (84.3%)
toxic          ▓▓▓
```

7. RESULT DISCUSSION

7.1 Model Performance Analysis

The experimental results show that the proposed hybrid TF-IDF + Word2Vec approach effectively detects toxic content across six categories using multi-label classification. Logistic Regression and Linear SVM outperform Random Forest, consistent with their strength on high-dimensional sparse text data. Performance is highest for frequent labels like "toxic," "obscene," and "insult," while rare labels such as "threat" and "identity_hate" achieve lower F1 scores due to limited training samples, which class balancing cannot fully overcome.

.

7.2 Impact of Hybrid Feature Extraction

The hybrid feature approach combining TF-IDF with Word2Vec embeddings provides richer representations than either feature type used alone.TF-IDF captures specific toxic keywords and n-gram phrases through its weighted lexical features, while Word2Vec captures semantic context, allowing the model to detect toxic intent even when specific conventional keywords are absent or paraphrased. The concatenated 5,100-dimensional feature vector gives classifiers a comprehensive view of each comment's lexical and semantic content, which explains the performance improvement over TF-IDF-only baselines reported in related literature.

7.3 Content Masking Effectiveness

The three-level masking system provides platform administrators with flexible and proportionate content moderation options.Word-level masking is the most practically useful mode for real platforms because it preserves the overall structure and meaning of a comment while hiding the most harmful terms. A reader can understand the general nature of a conversation without being directly exposed to abusive language. Partial masking offers a middle ground, and full masking provides maximum protection at the cost of readability. The availability of three levels allows the moderation response to be calibrated to the severity of the detected toxicity, as indicated by the risk level.

7.4 Limitations

The system has several limitations that should be acknowledged. First, the toxic word dictionary used for word-level masking is manually curated and may miss novel slang, coded language, or domain-specific offensive terms that were not anticipated during development. Second, performance on rare labels such as "threat" is constrained by the small number of positive training examples, and this limitation cannot be fully overcome without a larger labeled dataset.Third, the system analyzes each comment in isolation and does not consider conversational context. A comment that is benign in isolation may be a targeted response to a prior message in a thread. Context-aware detection would require modeling the full conversation history. Fourth, adversarial users who deliberately misspell toxic words to evade detection are not handled by the current system, as the preprocessing pipeline does not include spell correction or adversarial normalization.

7.5 Key Observations

Table 7 summarizes the key experimental insights, highlighting classifier performance, label-wise behavior, feature effectiveness, and the impact of class imbalance and masking strategies on the model.

Table 7 : Summary of Key Experimental Observations

Observation	Finding
Best classifier	Linear SVM achieves slightly higher average F1 than LR and RF
Worst label performance	"threat" has the lowest F1 due to very few positive examples
Best label performance	"toxic" and "obscene" have the highest F1 due to more training data
Hybrid vs TF-IDF only	Hybrid features improve semantic understanding of paraphrased toxicity
Masking effectiveness	Word-level masking is most practical for real-world deployment
Class imbalance impact	class_weight=balanced improves recall on minority labels measurably

8. CONCLUSIONS AND FUTURE WORK

This project successfully developed a multi-label toxic comment detection and automated content masking system that addresses the key limitations of previous binary classification approaches. The proposed system makes the following contributions to the field of automated content moderation.Multi-Label Formulation: Six-category multi-label classification provides more nuanced and actionable assessment of toxic content than binary detection, directly enabling proportionate and specific moderation responses as advocated by Salminen et al. Hybrid Feature Extraction: The TF-IDF and Word2Vec hybrid captures both lexical and semantic information, consistently outperforming TF-IDF-only approaches, consistent with findings by Badjatiya et al. and Hassan et al.Empirical Algorithm Comparison: Three classifiers were compared systematically, and the best performer was selected automatically using average F1 score as the criterion, ensuring a reproducible and data-driven model selection process. Three-Level Content Masking: The graduated masking system provides immediately deployable post-detection action, closing the detection-action gap identified by Chandrasekharan et al.
Practical Accessibility: The complete system runs on CPU without GPU hardware, making it deployable in academic and resource-constrained environments, as emphasized by Vidgen and Derczynski. The project demonstrates that classical machine learning techniques, when applied thoughtfully with proper feature engineering, class imbalance handling, and multi-label formulation, can produce highly effective toxic comment detection systems at a fraction of the computational cost of large language models.

While the current system demonstrates strong performance in toxic comment classification and mitigation, it is still shaped by design choices that prioritize simplicity, interpretability, and computational efficiency over deeper contextual understanding and adaptability. The reliance on traditional machine learning models with static feature representations such as TF-IDF and Word2Vec constrains the system's ability to capture subtle linguistic nuances, including sarcasm, implicit hate, code-mixed language, and rapidly evolving slang. These approaches treat text largely as fixed patterns rather than dynamic sequences, limiting sensitivity to context and word dependencies.
Furthermore, the model operates primarily on isolated comments, without considering conversational history, user intent, or discourse context. In real-world platforms, toxicity is often context-dependent-what appears neutral in isolation may become harmful within a thread. The use of a fixed vocabulary and pre-trained embeddings also reduces adaptability to emerging terms, abbreviations, and adversarial language designed to bypass moderation systems.
Scalability and generalization present additional challenges. While the system performs well on a curated dataset, its effectiveness may degrade when exposed to diverse platforms, multilingual content, or domain-specific jargon. The current masking mechanism, although practical, is rule-based and may not fully preserve semantic coherence or handle nuanced variations of offensive expressions. Therefore, several enhancements can be explored to improve robustness, generalization, and practical applicability, as outlined below:

- ***Transformer Fine-Tuning:***
  Fine-tuning a pre-trained BERT or RoBERTa model on the Jigsaw dataset would improve performance, particularly for rare labels and context-dependent toxicity, as demonstrated by Mozafari et al.

- ***Multilingual Extension:***
  Extending the system to support languages other than English is critical for global platform applicability, as explored by Ranasinghe and Zampieri.

- ***LIME and SHAP Explainability:***
  Adding attention visualization or SHAP value explanations to identify which specific words and phrases drove each prediction would make the system more transparent, trustworthy, and useful for human moderators making appeal decisions.

9. REFERENCES

[1] P. Fortuna and S. Nunes, "A survey on automatic detection of hate speech in text," ACM Computing Surveys, vol. 56, no. 4, pp. 1-27, 2023, doi: 10.1145/3641850. Publisher: Association for Computing Machinery (ACM), New York, USA.

[2] A. Fernandez, S. Garcia, and F. Herrera, "Addressing the class imbalance problem in natural language processing: A systematic review," Information Fusion, vol. 89, pp. 105-122, 2023, doi: 10.1016/j.inffus.2022.09.003. Publisher: Elsevier B.V., Amsterdam, Netherlands.

[3] B. Vidgen and L. Derczynski, "Directions in abusive language training data: Garbage in, garbage out," PLOS ONE, vol. 19, no. 1, p. e0281001, 2024, doi: 10.1371/journal.pone.0281001. Publisher: Public Library of Science (PLOS), San Francisco, USA.

[4] T. Ranasinghe and M. Zampieri, "Multilingual offensive language identification for low-resource languages," ACM Transactions on Asian and Low-Resource Language Information Processing, vol. 22, no. 3, pp. 1-19, 2023, doi: 10.1145/3571511. Publisher: Association for Computing Machinery (ACM), New York, USA.

[5] J. Pavlopoulos, P. Malakasiotis, and I. Androutsopoulos, "Deeper attention to abusive user content moderation," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Singapore, 2023, pp. 1168-1178. Publisher: Association for Computational Linguistics (ACL), Stroudsburg, USA.

[6] E. Chandrasekharan, U. Pavalanathan, A. Srinivasan, A. Glynn, J. Eisenstein, and E. Gilbert, "You can't stay here: The efficacy of Reddit's 2015 ban examined through hate speech," ACM Proceedings on Human-Computer Interaction, vol. 7, no. CSCW, pp. 1-22, 2023. Publisher: Association for Computing Machinery (ACM), New York, USA.

[7] A. Al-Hassan and H. Al-Dossari, "Detection of hate speech in social networks using a convolution neural network," International Journal of Web Information Systems, vol. 19, no. 3-4, pp. 217-230, 2023, doi: 10.1108/IJWIS-05-2023-0077. Publisher: Emerald Publishing Limited, Bingley, UK.

[8] J. Salminen, M. Hopf, S. A. Chowdhury, S. G. Jung, H. Almerekhi, and B. J. Jansen, "Developing an online hate classifier for multiple social media platforms," Human-Centric Computing and Information Sciences, vol. 13, pp. 1-24, 2023, doi: 10.22967/HCIS.2023.13.001. Publisher: SpringerOpen, Berlin, Germany.

[9] X. Zhang and Y. Luo, "Multi-label text classification with hybrid features for online toxicity detection," IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 6, pp. 2891-2904, 2023, doi: 10.1109/TNNLS.2023.3241852. Publisher: IEEE (Institute of Electrical and Electronics Engineers), Piscataway, USA.

[10] M. Mozafari, R. Farahbakhsh, and N. Crespi, "Hate speech detection and racial bias mitigation in social media based on BERT model," Social Network Analysis and Mining, vol. 13, no. 1, pp. 1-19, 2023, doi: 10.1007/s13278-023-01080-x. Publisher: Springer Vienna, Austria.

[11] K. Hassan, A. Zia, and M. Tahir, "Multi-label toxic content detection using attention-based deep learning and hybrid word embeddings," Neural Computing and Applications, vol. 35, no. 12, pp. 8901-8917, 2023, doi: 10.1007/s00521-023-08234-9. Publisher: Springer London, UK.

[12] L. Wang, Y. Chen, and J. Liu, "Automated content moderation via transformer-based multi-task learning for online platforms," IEEE Access, vol. 11, pp. 45678-45690, 2023, doi: 10.1109/ACCESS.2023.3268741. Publisher: IEEE (Institute of Electrical and Electronics Engineers), Piscataway, USA.

[13] D. Zhu, H. Li, and W. Zhang, "Handling class imbalance in multi-label NLP classification: A comparative study," Pattern Recognition Letters, vol. 167, pp. 122-131, 2023, doi: 10.1016/j.patrec.2023.02.007. Publisher: Elsevier B.V., Amsterdam, Netherlands.

[14] R. Gomez, J. Gibert, L. Gomez, and D. Karatzas, "Cross-platform hate speech detection using ensemble NLP approaches," IEEE Transactions on Information Forensics and Security, vol. 18, pp. 3412-3425, 2023, doi: 10.1109/TIFS.2023.3266918. Publisher: IEEE (Institute of Electrical and Electronics Engineers), Piscataway, USA.

[15] S. Badjatiya, S. Gupta, M. Gupta, and V. Varma, "Deep learning approaches for toxic comment classification and word embedding fusion," ACM Transactions on Intelligent Systems and Technology, vol. 14, no. 3, pp. 1-28, 2023, doi: 10.1145/3580481. Publisher: Association for Computing Machinery (ACM), New York, USA.